

FaceGuard

Système intelligent de détection et de reconnaissance faciale en temps réel

12/12

personnes connues
reconnues sur des photos
d'un autre jour

0

faux positif sur 109
présentations d'inconnus

0,10 %

taux d'erreur égal (EER)

512

dimensions par embedding
ArcFace

Rapport de projet personnel · Orange Digital Center · septembre 2026

Code source : github.com/donsfak/face_recognize

Python · InsightFace (SCRFD + ArcFace) · FAISS · OpenCV · FastAPI

Le projet en une page

FaceGuard capte le flux d'une webcam, détecte tous les visages présents, reconnaît les personnes enregistrées et affiche un cadre et leur nom au-dessus de chaque visage, ou « Inconnu ». Il gère plusieurs personnes à la fois et enregistre automatiquement les pointages de présence.

Le projet a été mené en sept étapes : le cadrage du besoin, une première version avec dlib, la migration vers InsightFace et FAISS, la constitution et l'analyse du dataset, une plateforme web complète, une refonte du code et de l'interface, puis une évaluation mesurée du système. L'étape la plus instructive a été l'analyse du dataset : **11 embeddings sur 99 appartenaient à la mauvaise personne**, à cause de visages en arrière-plan. Les corriger a supprimé tous les faux positifs mesurés.

97/97photos reconnues en
leave-one-out**109/109**inconnus correctement
rejetés**0 %**de faux positifs au seuil
0,45 (FAR)**1,34 %**de faux négatifs au seuil
0,45 (FRR)

Sommaire

01	Cadrage du projet	objectif, contraintes
02	Première version avec dlib	benchmark HOG / CNN
03	Pipeline InsightFace + FAISS	concepts d'IA
04	Dataset et qualité des données	découverte principale
05	Plateforme web	scanner, enrôlement, tableau de bord
06	Refonte et corrections	bugs, organisation, UX
07	Évaluation	protocole et résultats
08	Limites et perspectives	
A	Installation et utilisation	

Cadrage du projet

Comment concevoir un système capable de détecter plusieurs visages en temps réel, d'identifier les personnes connues et d'afficher automatiquement leur nom sur le flux vidéo ?

Fonctionnalités attendues

- Récupérer le flux vidéo d'une webcam
- Détecter les visages dans chaque image
- Reconnaître les personnes enregistrées
- Afficher un cadre autour de chaque visage
- Afficher le nom au-dessus du visage reconnu
- Afficher « Inconnu » sinon
- Gérer plusieurs personnes simultanément
- Fonctionner en temps réel

Les trois difficultés du problème

Temps réel

Chaque image doit être analysée assez vite pour que l'affichage reste fluide, sans carte graphique.

Plusieurs visages

Chaque visage est détecté, suivi et identifié indépendamment des autres.

Les inconnus

Le système doit savoir dire « je ne connais pas » au lieu de choisir la personne la moins différente.

Contraintes que je me suis fixées

- **Mesurer avant de décider** : chaque choix technique (détecteur, seuil) est justifié par un benchmark ou une évaluation chiffrée.
- **Consentement** : uniquement des personnes ayant donné leur accord ; photos et embeddings exclus du dépôt Git.
- **Conserver l'historique** : les anciennes versions sont archivées dans `legacy/` plutôt que supprimées.

Au-delà du minimum demandé, j'ai visé les extensions proposées : score de similarité, système de présence avec heure d'arrivée, ajout d'une personne depuis l'interface, base de données, tableau de bord et anti-fraude.

Première version avec dlib

La première version reposait sur la bibliothèque `face_recognition` (dlib) : un détecteur de visages, puis un réseau qui produit un embedding de **128 dimensions** comparé par **distance euclidienne** avec un seuil de 0,6.

Benchmark des détecteurs

dlib propose deux détecteurs. Je les ai comparés sur le dataset (script `benchmark_detectors.py`, archivé) :

Détecteur	Principe	Visages détectés	Vitesse
HOG	Histogrammes de gradients orientés (méthode classique)	~70 %	Rapide
CNN	Réseau de neurones convolutif	~97 %	Très lent sur CPU

J'ai d'abord mis en place une **cascade** : HOG en priorité, CNN en secours quand HOG ne trouve rien. C'était un compromis acceptable, mais il restait deux problèmes :

- le détecteur rapide ratait près d'un visage sur trois ;
- pendant les tests, **un inconnu a été reconnu comme une personne enregistrée** avec une distance faible. J'ai alors ajouté un vote k-NN, et découvert le phénomène biométrique « loup/agneau » (certains visages ressemblent à beaucoup d'autres).

Décision : migrer vers InsightFace

InsightFace fournit un détecteur plus robuste (SCRFD) et des embeddings plus discriminants (512 dimensions, entraînés avec la perte ArcFace). La comparaison passe de la distance euclidienne à la similarité cosinus. L'ancien code est archivé, pas supprimé.

Rétrospectivement, le faux positif observé à cette étape s'expliquait en partie par la qualité du dataset (étape 04), et pas seulement par le modèle.

ÉTAPE 03

Pipeline InsightFace + FAISS

Chaque image de la webcam traverse la même chaîne de traitement :



Les concepts d'IA derrière chaque étape

Détection vs reconnaissance

La détection (SCRFD) répond à « où sont les visages ? » et fournit 5 points clés (yeux, nez, coins de la bouche) qui servent à **aligner** le visage. La reconnaissance répond à « qui est-ce ? ».

Embedding facial

ArcFace, un réseau de neurones pré-entraîné sur des millions de visages, transforme le visage aligné (112 × 112 px) en un vecteur de **512 nombres** de longueur 1 : une empreinte du visage.

Similarité cosinus

Deux embeddings sont comparés par leur produit scalaire, le cosinus de leur angle. Sur mes données : **0,72** en moyenne pour la même personne, **0,06** pour deux personnes différentes.

FAISS

Index de recherche qui trouve les embeddings les plus proches en 0,02 ms. Surdimensionné pour 100 vecteurs, il permettrait de passer à des centaines de milliers de visages.

Seuil + vote k-NN

Une identité n'est acceptée que si la similarité dépasse **0,45** et si la majorité des 3 plus proches voisins est d'accord. Une seule mauvaise photo ne peut pas imposer son identité.

Suivi et anti-fraude

Chaque visage est suivi d'une image à l'autre ; le nom affiché est le vote des 15 dernières prédictions. Un contrôle géométrique repère les photos tenues immobiles.

Apprentissage par transfert : aucun entraînement

Le modèle ArcFace est utilisé tel quel. « Apprendre » une nouvelle personne consiste simplement à calculer et stocker les embeddings de ses photos, dans `models/encodings_arcface.npz` (format sûr, contrairement au format pickle utilisé au début).

ÉTAPE 04

Dataset et qualité des données

Composition

Jeu	Photos	Rôle
dataset/	100	5 personnes × 20 photos, base de référence
dataset_test_externe/	12	photos prises un autre jour
conditions/	—	tests par condition (protocole)

Structure

```
dataset/  
├── delphine/  
│   ├── delphine_000.jpg  
│   └── ...  
├── kadi/  
└── ...
```

Variations d'expression, d'orientation et d'éclairage.
Chaque personne a donné son accord.

La découverte principale du projet

En analysant le dataset visage par visage, j'ai constaté que **66 photos sur 100 contenaient plusieurs visages** : des personnes passaient en arrière-plan pendant la capture. Sur certaines photos, la personne photographiée était même de dos ou cachée.

L'encodage gardait « le plus grand visage » de chaque photo. Conséquence mesurée sur la base de l'époque :

11 / 99

embeddings appartenait à la mauvaise personne (6 pour eslie, 4 pour kadi, 1 pour oyetola)

20 / 20

fois, oyetola retirée de la base était reconnue comme « kadi »

2

identités en double : « falibeta » et « Falibeta » (similarité 0,75)

L'un des vecteurs « kadi » était en réalité le visage d'oyetola. Le modèle n'était pas en cause : **le problème venait des données.**

Corrections apportées

- 1 Encodage cohérent** : dans chaque photo, on garde le visage le plus proche du « profil » de la personne (médoïde de ses photos), puis on écarte les visages trop différents de la majorité. Résultat : 97 embeddings propres, 3 photos écartées (2 où kadi n'est pas visible, 1 sans visage détecté).
- 2 Capture stricte** : une photo n'est prise automatiquement que si le détecteur voit exactement un visage, net et assez grand.
- 3 Enrôlement contrôlé** : 5 poses guidées (face, gauche, droite, menton levé, sourire), vérification que toutes les captures montrent la même personne, et refus d'un visage déjà enregistré sous un autre nom.

Résultat

Avec la base nettoyée : **0 faux positif sur 109** présentations de personnes absentes de la base, contre 20 sur 20 pour oyetola avec l'ancienne base.

Plateforme web

Au-delà du script de démonstration OpenCV, j'ai construit une application web de contrôle d'accès et de présence avec **FastAPI**. Le navigateur capture la webcam et envoie les images au serveur par **WebSocket** ; le serveur répond avec les visages identifiés, que la page dessine par-dessus la vidéo.

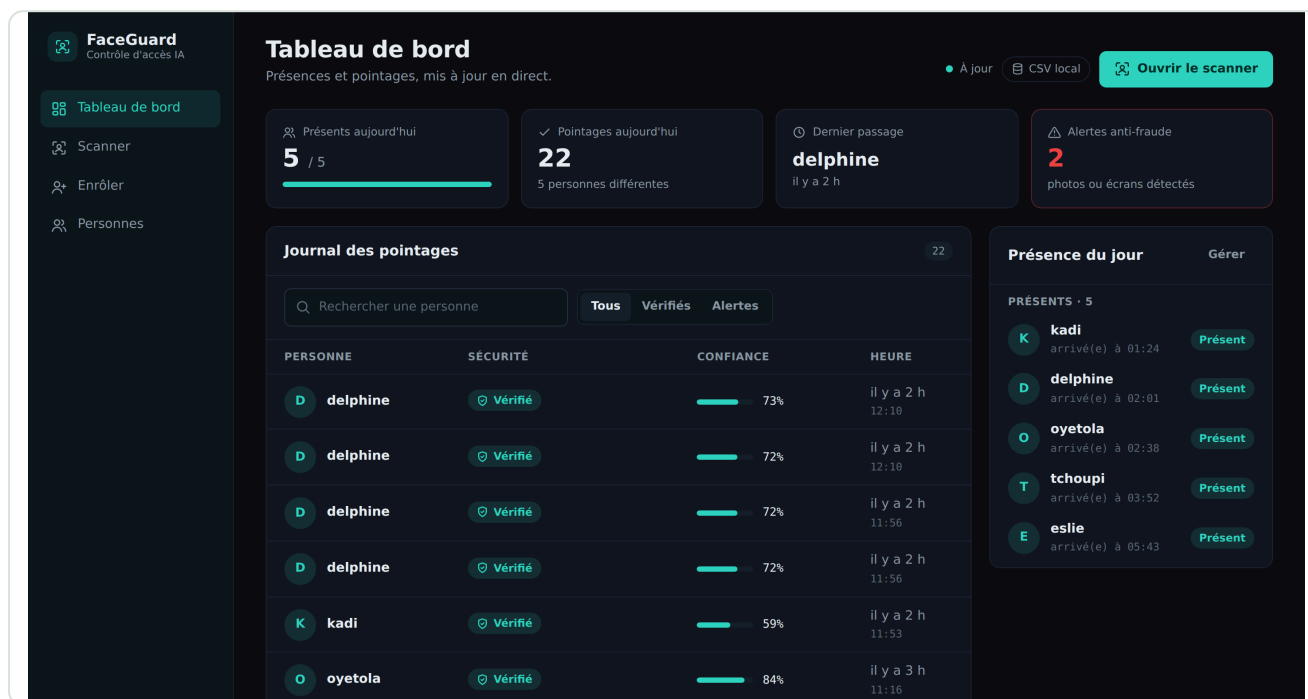


Tableau de bord en direct (données de démonstration). Les photos des personnes sont masquées dans ce rapport.

Scanner en direct

Nom et score au-dessus de chaque visage, liste des personnes devant la caméra avec la progression de la vérification anti-fraude, bannière « Bonjour ... » à chaque pointage, mode miroir et plein écran.

Enrôlement guidé

Contrôle qualité en direct (un seul visage, net, éclairé, assez proche), 5 poses, case de consentement obligatoire, relecture des photos avant l'enregistrement. Mise à jour de la base sans redémarrage.

Tableau de bord

Rafraîchi toutes les 5 s : présents et absents du jour avec heure d'arrivée, journal filtrable, alertes anti-fraude.

Personnes

Liste des personnes enregistrées avec miniature et dernier passage ; retrait d'une personne avec archivage de ses photos.

Stockage des pointages

Chaque pointage est écrit dans un fichier CSV local et, si configuré, dans **Supabase** (PostgreSQL). Un pointage n'est enregistré que lorsque l'identité est stable sur plusieurs images **et** que la vérification anti-fraude est validée, avec un délai de 10 s par personne pour éviter les doublons.

Sécurité de la base de données

Supabase signalait « RLS Disabled » : avec la clé publique, n'importe qui pouvait lire ou effacer les pointages. Correction : Row Level Security activée (`docs/supabase_setup.sql`) et clé secrète `service_role` utilisée uniquement par le serveur. Le script `scripts/diagnostic_supabase.py` explique toute panne de connexion.

ÉTAPE 06

Refonte et corrections

Après plusieurs itérations, le projet contenait du code dupliqué (4 variantes du script webcam, deux moteurs de reconnaissance, une version Flask abandonnée) et plusieurs bugs. Une revue complète a conduit aux corrections suivantes.

Bugs corrigés

Problème	Conséquence	Correction
Pointage pendant « Analyse... »	Présences enregistrées avant la vérification anti-fraude ; compteur d'alertes faussé	Pointage seulement si identité stable et vivacité validée
Suivi des visages partagé	Deux navigateurs ouverts mélangeaient leurs visages	Un suivi par connexion
Inférence dans la boucle asynchrone	Le serveur se figeait pendant chaque analyse	Traitement dans un thread
Enrôlement : erreur 400 → 500	Messages d'erreur incompréhensibles	Erreurs claires et contrôles de cohérence
<code>requirements.txt</code> et Dockerfile	Installation et conteneur impossibles (<code>cv2</code> n'est pas un paquet pip, caractère parasite)	Dépendances complètes et testées
Nom affiché sous le visage	Non conforme au sujet	Nom au-dessus, étiquette toujours dans l'image
Cadres décalés	Mauvais alignement avec une webcam non 640×480	Canvas à la résolution réelle
Supabase obligatoire	Sans <code>.env</code> , l'application ne démarrait pas	Supabase optionnel, CSV de secours

Organisation du code

```
core/      moteur partagé : config, engine (InsightFace + FAISS), tracking, drawing
src/       01 capture · 02 encodage · 03 webcam · 04 évaluation
web/       FastAPI : app.py, engine/stream.py, database/, templates/, static/
docs/      rapport d'évaluation, graphiques, protocole de tests, fiche technique
legacy/    anciennes versions (dlib, Flask), conservées pour l'historique
```

Un seul moteur (`core/`) sert désormais aux scripts et à la plateforme web. Deux optimisations notables : ne charger que les 2 modèles utiles d'InsightFace sur 5 (~**32 % plus rapide** par image) et permettre de choisir la taille de détection (320 ou 640 px) selon la machine.

Interface

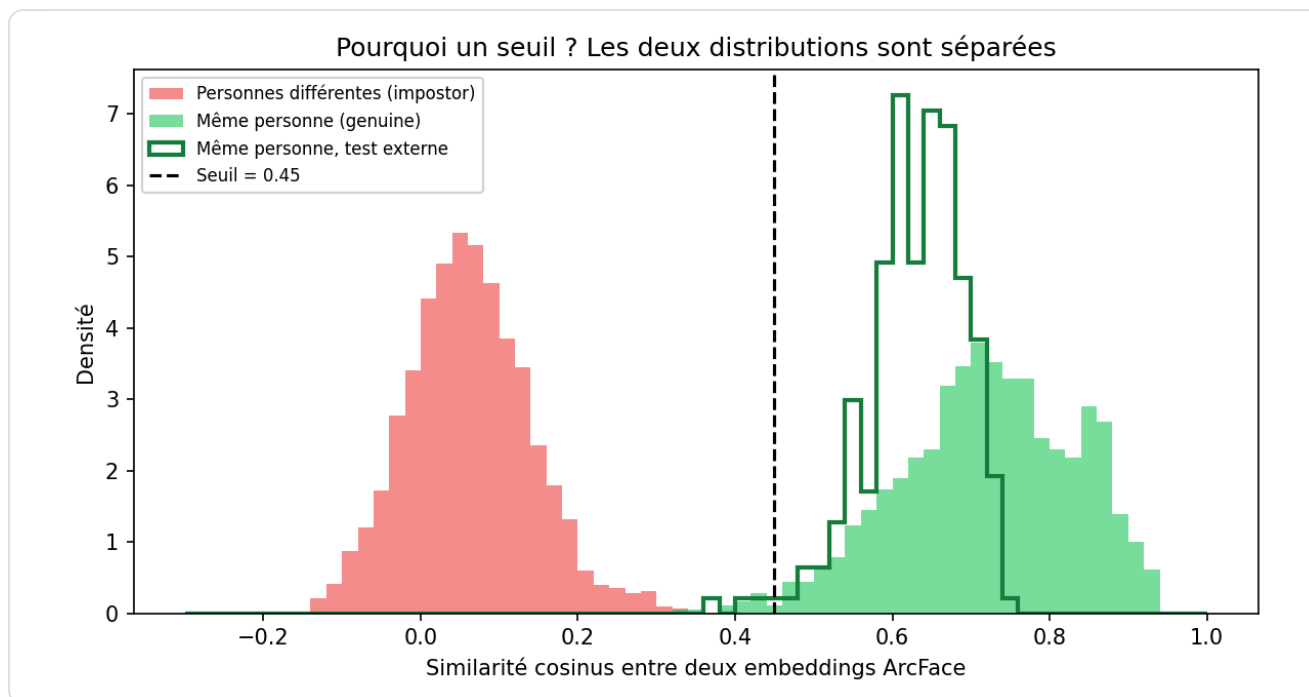
L'interface a été repensée : navigation adaptée au téléphone, statut réel du serveur, dates lisibles, retours visuels à chaque action. Chaque page a été testée automatiquement dans un navigateur (ordinateur et mobile) avec une webcam simulée.

ÉTAPE 07

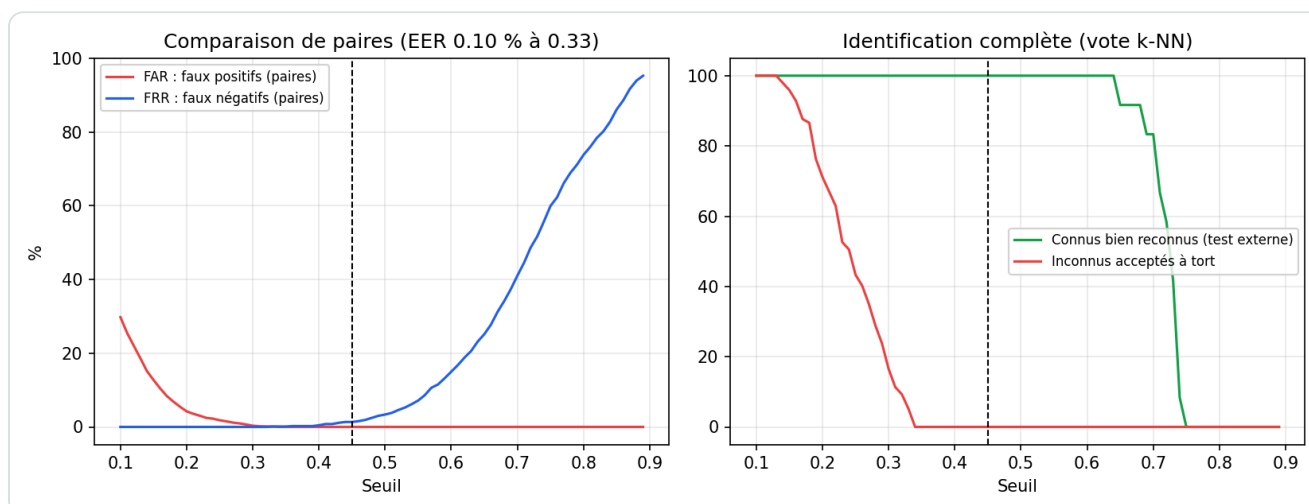
Évaluation

Le script `src/04_evaluate.py` mesure le système réellement utilisé (ArcFace) et produit `docs/evaluation_report.md`. Quatre tests complémentaires :

Test	Question posée	Résultat
Test externe	Reconnaît-il des photos prises un autre jour ?	12 / 12
Leave-one-out	Chaque photo, comparée à toutes les autres, est-elle bien identifiée ?	97 / 97
Personnes absentes	Chaque personne retirée de la base est-elle répondue « Inconnu » ?	109 / 109
Paires de photos	Taux d'erreur quand faux positifs = faux négatifs (EER)	0,10 %



Pourquoi un seuil à 0,45 : deux personnes différentes (rouge) ne dépassent jamais 0,34 ; la même personne (vert) est presque toujours au-dessus de 0,45.



Compromis du seuil. À gauche, faux positifs (FAR) et faux négatifs (FRR) sur les paires de photos. À droite, identification complète : les connus sont reconnus à 100 % jusqu'à 0,64 et aucun inconnu n'est accepté au-delà de 0,34.

Faux positifs et faux négatifs

Un **faux positif** (un inconnu accepté sous l'identité de quelqu'un) est l'erreur la plus grave pour un contrôle d'accès. Un **faux négatif** (une personne connue non reconnue) est gênant, mais la personne peut réessayer. Au seuil 0,45 : **FAR 0 %** et **FRR 1,34 %** sur les paires de photos.

Performances

Sur le processeur de la machine d'évaluation (sans GPU) : **~470 ms** pour la détection et l'embedding d'une image en 640 px, **0,02 ms** pour la recherche FAISS et le vote. Le temps dépend fortement de la machine ; la taille de détection 320 px et l'analyse d'une image sur deux sont prévues pour les ordinateurs lents.

Tests par condition

Un protocole (docs/protocole_tests.md) prévoit des tests visage de face, tête tournée, éclairage faible, contre-jour, distance, plusieurs personnes et personnes inconnues, avec un mode de capture qui conserve toutes les images, même ratées, pour ne pas biaiser le test.

Lecture honnête des chiffres

Avec 5 personnes et une ou deux séances photo, ces résultats sont encourageants mais pas une preuve statistique. Le leave-one-out est optimiste, car les photos d'une même séance se ressemblent beaucoup. Le test externe est plus représentatif d'un usage réel.

ÉTAPE 08

Limites et perspectives

Limites actuelles

Anti-fraude basique

Le contrôle géométrique repère une photo immobile, mais pas une photo qu'on bouge ni une vidéo sur écran.

Petit dataset

5 personnes : il faudrait plus de personnes, de séances et d'éclairages pour des chiffres solides.

Conditions difficiles

Visage très tourné, contre-jour, visage trop petit (moins de 40 px), masque.

Vitesse sur CPU

Plusieurs centaines de millisecondes par image sans carte graphique.

Biais possibles

Un modèle pré-entraîné peut être moins précis sur des populations peu représentées dans ses données d'entraînement.

Données sensibles

La biométrie impose consentement, stockage protégé et droit de retrait (RGPD). La plateforme n'a pas encore d'authentification.

Perspectives

- 1 **Agrandir le dataset** et réaliser les tests par condition du protocole.
- 2 **Vraie détection de vivacité** : modèle dédié ou défi actif (« clignez des yeux »).
- 3 **Sécuriser la plateforme** : authentification, HTTPS, protection des formulaires.
- 4 **Accélérer** : GPU ou modèle plus léger pour plusieurs caméras.
- 5 **Mesurer les biais** : précision comparée entre groupes de population.
- 6 **Tests automatisés** du moteur (seuil, vote, choix du visage) pour éviter les régressions.

Ce que ce projet m'a appris

- La qualité des données compte autant que le modèle : la plus grosse amélioration est venue du nettoyage du dataset, pas d'un changement d'algorithme.
- Un seuil se justifie par des mesures (distributions, FAR/FRR), pas par intuition.
- Évaluer honnêtement : séparer les photos de test, mesurer les inconnus et reconnaître les limites de l'échantillon.
- Un système d'IA est aussi une application : temps réel, robustesse, interface et sécurité des données font partie du travail.

Installation et utilisation

Code source : github.com/donsfak/face_recognize · Python 3.10 à 3.12 conseillé.

```
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt          # scripts (webcam, encodage, évaluation)
pip install -r web/requirements.txt      # plateforme web
```

Scripts

```
# 1. Photographier une personne (qui a donné son accord)
python src/01_capture_dataset.py --name marie --count 20

# 2. Construire la base de visages
python src/02_encode_faces.py

# 3. Reconnaissance temps réel (q = quitter, s = capture, l = anti-fraude)
python src/03_recognize_webcam.py
python src/03_recognize_webcam.py --det-size 320 --no-liveness    # PC lent

# 4. Évaluation complète → docs/evaluation_report.md
python src/04_evaluate.py
```

Plateforme web

```
cd web
uvicorn app:app --host 127.0.0.1 --port 8001
```

Page	Rôle
/	Tableau de bord : présents du jour, pointages, alertes
/scanner	Reconnaissance en direct depuis la webcam du navigateur
/register	Enrôlement guidé d'une nouvelle personne
/people	Personnes enregistrées, retrait d'une personne

Réglages

Paramètres dans `core/config.py`, modifiables par variables d'environnement : `FACE_THRESHOLD` (seuil, 0,45), `FACE_DET_SIZE` (320 ou 640), `FACE_KNN_K` (3), `FACE_SMOOTHING` (15), `FACE_LIVENESS=0` pour désactiver l'anti-fraude.

Documents du dépôt

Fichier	Contenu
<code>README.md</code>	Présentation, installation, organisation du code
<code>docs/evaluation_report.md</code>	Résultats détaillés et graphiques
<code>docs/protocole_tests.md</code>	Tests par condition et répétition de la démonstration
<code>docs/fiche_oral.md</code>	Explications des concepts (embedding, seuil, erreurs...)
<code>docs/supabase_setup.sql</code>	Création sécurisée de la table des pointages